Progress Report

Day 2 — Frontend & Backend Foundation Setup
Task Management MERN Stack Platform
Shafin Nigamana | 20 May 2026

1. Day-2 Objective

The goal for Day-2 was to initialize both frontend and backend applications, organize the project structure, configure development tooling, and establish frontend-to-backend API communication.

2. Tasks Completed

Frontend Setup

1. Initialized React application using Vite inside the /client workspace
2. Installed frontend dependencies:

- react-router-dom
- axios

Figure 2.1 — React frontend Dependencies from Package.json

3. Removed default Vite boilerplate files and Created frontend folder structure:

- /components
- /pages
- /routes
- /services

- /styles

4. Created centralized Axios API service configuration

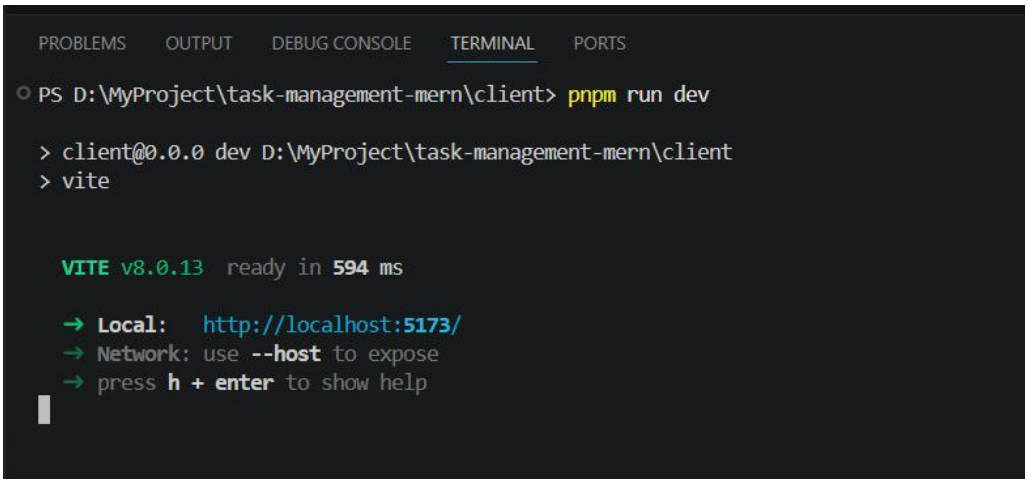

Figure 2.2 — React frontend initialized using Vite.

Backend Setup

- Initialized Express.js backend application inside /server
- Installed backend dependencies:

  - express
  - dotenv
  - cors
  - helmet
  - morgan
  - mongoose
  - mysql2
  - bcrypt
  - jsonwebtoken

- Configured backend using ES Modules (import/export syntax)
- Created backend MVC folder structure:

  - /controllers
  - /routes
  - /models
  - /middleware
  - /config

- Configured middleware:

- CORS
- Helmet
- Morgan
- express.json()

Figure 2.3 — Express backend Dependencies from Package.json

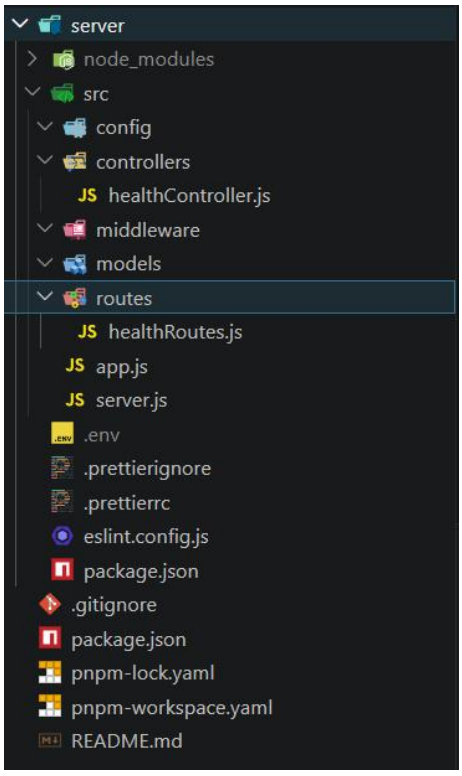

Figure 2.4 — Express backend initialized with MVC folder structure.

API Health Endpoint

- Created health controller and route structure
- Implemented /api/health endpoint
- Connected routes through centralized Express app configuration
- Verified backend API response successfully in browser

Current API response:


```
{
 "status": "OK",
 "message": "API is healthy"
}
```


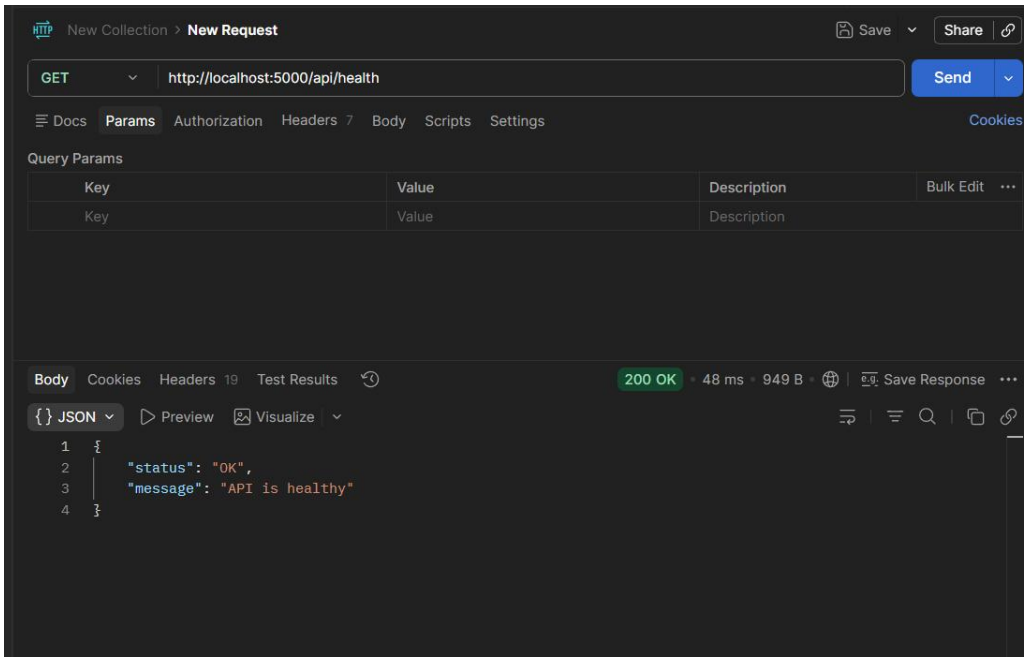

Figure 2.5 — API health endpoint response from backend server.

Frontend ↔ Backend Integration

- Configured Axios service using environment variables
- Connected frontend application with backend API
- Fetched API health response using React useEffect()
- Verified frontend rendering of backend API response

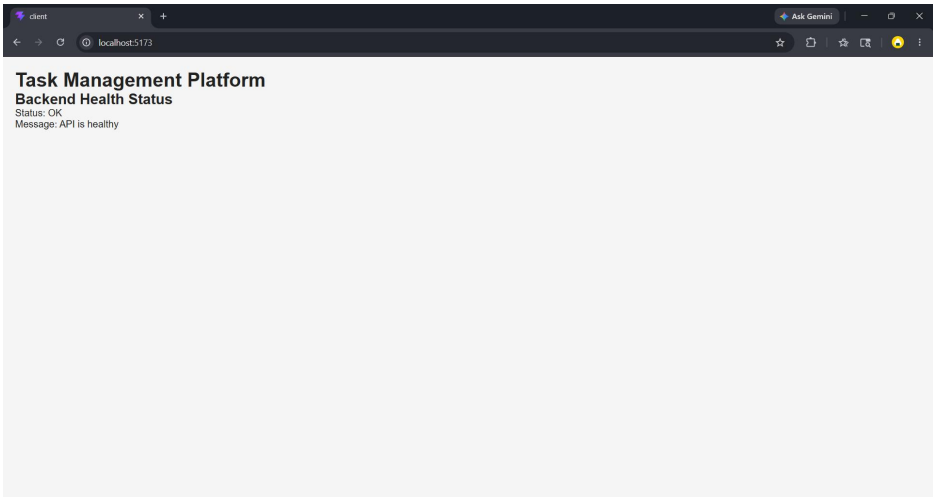

Figure 2.6 — Frontend successfully receiving backend API response.

ESLint & Prettier Configuration

- Configured ESLint for backend linting and code validation
- Configured Prettier for formatting consistency
- Removed unnecessary React rules from backend ESLint configuration
- Applied formatting across backend source files

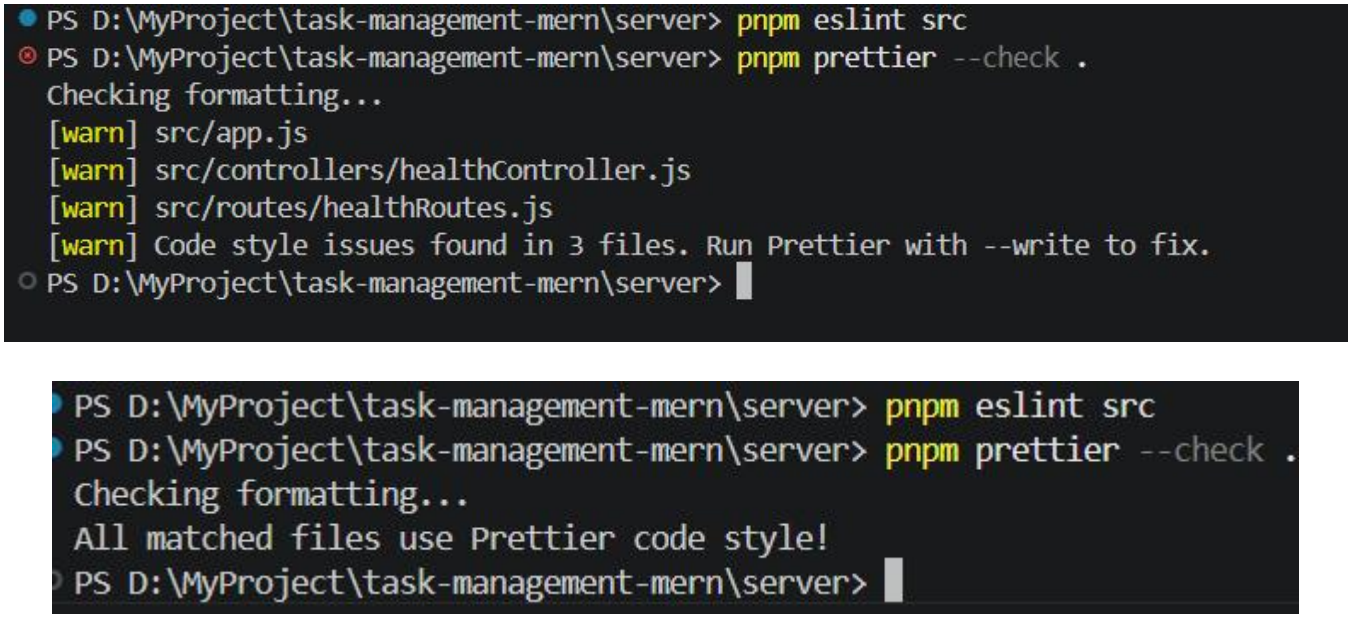

Figure 2.7 — ESLint and Prettier configuration completed successfully.

Workspace & Repository Cleanup

- Configured pnpm monorepo workspace using pnpm-workspace.yaml
- Added root-level package.json scripts:
  - dev:client
  - dev:server
- Updated .gitignore for monorepo dependency handling
- Removed unused boilerplate and placeholder files
- Verified node_modules were not tracked in Git

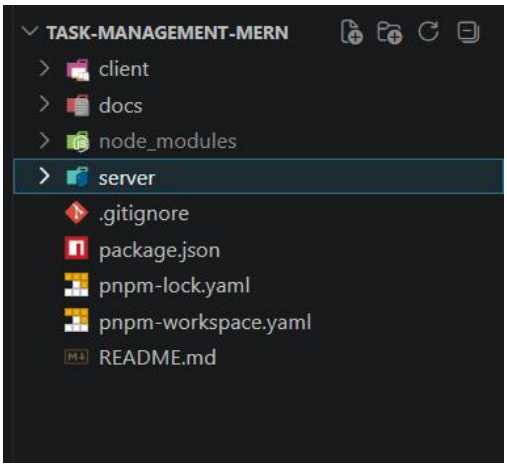

Figure 2.8 — Updated monorepo repository structure after cleanup.

Git Workflow

- Committed frontend and backend foundation setup using feature branch workflow
- Pushed changes to remote GitHub repository
- Created cleanup commit for repository standardization

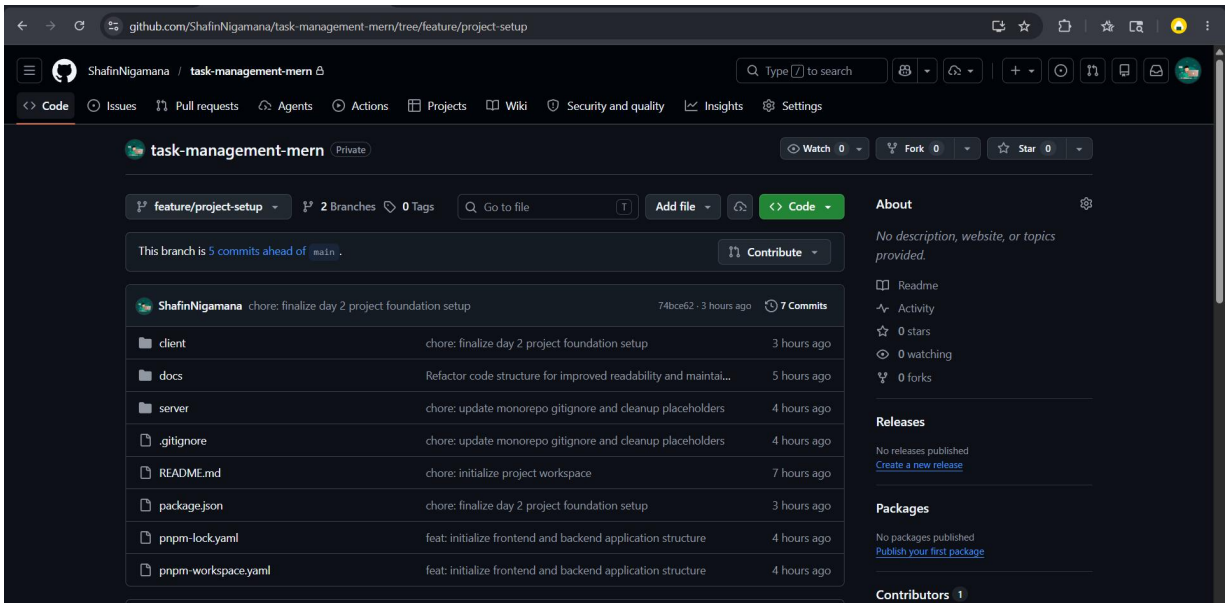

Figure 2.9 — Git commits pushed successfully to remote repository.

3. Current Repository Structure

Current project structure after Day 2:

```
task-management-mern/
├── client/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── routes/
│   │   ├── services/
│   │   └── styles/
│   │
│   ├── package.json
│   └── vite.config.js
│
├── docs/
│   ├── architecture/
│   └── daily-logs/
│
├── server/
│   ├── src/
│   │   ├── config/
│   │   ├── controllers/
│   │   ├── middleware/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── app.js
│   │   └── server.js
│   │
│   ├── package.json
```

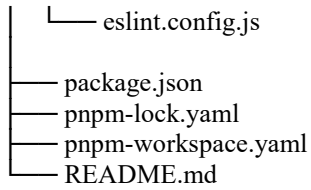

Figure 3.1 — Current workspace structure after frontend and backend initialization.

4. Challenges Faced

- Backend ESLint configuration initially included React rules and required cleanup for backend-only usage
- Additional .gitignore updates were required after pnpm workspace dependencies were generated
- Frontend environment variables required VITE_ prefix formatting for compatibility with Vite
- Frontend and backend servers required separate terminal execution before root scripts were configured

5. Next Steps (Day 3)

- Configure MongoDB Atlas connection using Mongoose
- Configure MySQL database connection using mysql2
- Create centralized database configuration modules
- Extend /api/health endpoint to verify database connection status
- Begin schema planning for Users and Teams

6. Conclusion

Day 2 focused on building the frontend and backend foundation of the application. Both applications are now initialized successfully, API communication is functioning correctly, and the monorepo workspace structure is standardized for future development.